ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG DẠI HỌC SƯ PHẠM
KHOA TOÁN – TIN



BÁO CÁO BÀI TẬP THỰC HÀNH NHÓM SỐ 1

Học phân: Lập trình mạng

ĐỀ TÀI:

PHÂN TÍCH ỨNG DỤNG MẠNG YOUTUBE

Nhóm thực hiện	:	09	
Thành viên	:	1. Phimmasone Khamphouvanh	: 3120224192
		2. Bounsavath Ameena	: 3120224190
		3. Lattanaphimphone Chindaphet	: 3120224191
		4. Silaphet Thit	: 3120224188
		5. Thepsoulivong Syvone	: 3120224193
Giảng viên hướng dẫn	:	Nguyễn Hoàng Hải	

Đà Nẵng,tháng 9 năm 2026

MỤC LỤC

Phần A – Phân tích ứng dụng mạng Youtube 3
A.1. Giới thiệu ứng dụng mạng Youtube 3
A.2. Mô hình hoạt động 3
A.2.1. Kiến trúc tổng thể 3
A.2.2. Các thành phần chính 3
A.2.3. Sơ đồ luồng thông điệp ((Sequence Diagram rút gọn) 4
A.3. Các giao thức mạng được sử dụng 4
A.3.1. Danh sách các giao thức 4
A.3.2. Giao thúc tầng giao vận: TCP và UDP 5
A.4. Kiểu tương tác của ứng dụng 5
A.4.1. Streaming và Adaptive Bitrate 5
A.4.2. Kiến trúc server và khả năng phục vụ đồng thời 6
A.5. Quan sát thực tế 6
A.5.1. Quan sát bằng netstat 6
A.5.2. Quan sát dns bằng wireshark 7
A.5.3. Quan sát HTTPS/TLS bằng wireshark 7
A.5.4. Nhận xét chung 7
Phần B - Thiết kế giao thức cho ứng dụng chat của nhóm 8
B.1. Mô tả ứng dụng 8
B.2. Kiến trúc 8
B.3. Đặc tả thông điệp 8
B.4. Kịch bản trao đổi 9
B.5. Quy tắc trạng thái và lỗi 10
Phần C - Thiết lập môi trường Python và chương trình đầu tiên 10
C.1. Môi trường thực hành 10
C.2. Chạy chương trình `hello_net.py` 11
C.3. So sánh kết quả với `ipconfig` 12
C.4. Chạy chương trình `dns_subnet.py` 13
C.5. Kiểm tra bằng netstat -an 14
Phần D – Câu hỏi thảo luận 14
KẾT LUẬN 15
TÀI LIỆU THAM KHẢO 17
PHỤ LỤC 18

Phần A – Phân tích ứng dụng mạng Youtube

A.1. Giới thiệu ứng dụng mạng Youtube

YouTube là nền tảng chia sẻ và phát video trực tuyến thuộc Google, cho phép người dùng tìm kiếm, xem, tải lên và tương tác với các nội dung video thông qua Internet. Người dùng có thể truy cập YouTube bằng nhiều phương thức khác nhau:

- Trình duyệt web trên máy tính (youtube.com)
- Ứng dụng di động (Android / iOS)
- Smart TV, máy console và các thiết bị khác

Trong bài tập này, nhóm lựa chọn phân tích quá trình người dùng truy cập và xem video YouTube trên trình duyệt web. Nội dung phân tích tập trung vào:

- Mô hình hoạt động của hệ thống (Client-Server, P2P hay Hybrid)
- Các giao thức mạng được sử dụng ở tầng ứng dụng và tầng giao vận
- Kiểu tương tác giữa client và server
- Quan sát lưu lượng mạng thực tế bằng công cụ netstat và Wireshark
- Mục tiêu của phần này là làm rõ cách một ứng dụng video trực tuyến quy mô lớn sử dụng các thành phần mạng và giao thức khác nhau để cung cấp nội dung cho hàng triệu người dùng đồng thời.

A.2. Mô hình hoạt động

A.2.1. Kiến trúc tổng thể

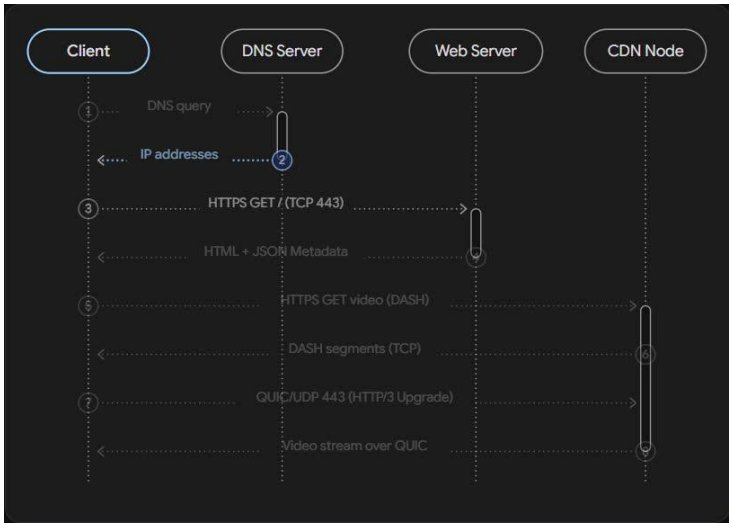
YouTube hoạt động chủ yếu theo mô hình Client-Server kết hợp với hệ thống CDN (Content Delivery Network) phân tán. Đây là mô hình trong đó:

- Client gửi yêu cầu đến các máy chủ của YouTube để lấy metadata, danh sách phát, thông tin tài khoản.
- Nội dung video được phân phối thông qua các máy chủ CDN đặt tại nhiều vị trí địa lý khác nhau.
- Ở cấp độ phân phối nội dung, đây có thể coi là kiến trúc Hybrid, vì hệ thống kết hợp giữa máy chủ trung tâm, các dịch vụ API và các máy chủ biên CDN.

A.2.2. Các thành phần chính

Thành phần	Vai trò
Client	Trình duyệt hoặc ứng dụng YouTube trên thiết bị người dùng
Web/API Server	www.youtube.com, youtubei.googleapis.com - xử lý yêu cầu trang web, tìm kiếm, metadata và dữ liệu người dùng
DNS Server	8.8.8.8 hoặc DNS của ISP - phân giải tên miền thành địa chỉ IP
CDN/Edge Server	googlevideo.com, *.c.youtube.com (Google Global Cache) - phân phối các đoạn video từ máy chủ gần người dùng
Authertication Server	accounts.google.com - xử lý xác thực tài khoản khi người dùng đăng nhập

A.2.3. Sơ đồ luồng thông điệp ((Sequence Diagram rút gọn)



Hình A.2.3. Hoạt động của YouTube kết hợp Client–Server và CDN

Diễn giải luồng:

1. Khi người dùng truy cập YouTube, client trước tiên thực hiện phân giải tên miền qua DNS để xác định địa chỉ IP của dịch vụ cần kết nối.
2. Sau đó, client gửi yêu cầu HTTPS GET đến máy chủ web hoặc API để lấy HTML và metadata của video.
3. Khi người dùng bắt đầu phát video, các yêu cầu tải dữ liệu được chuyển đến máy chủ CDN hoặc edge server gần vị trí địa lý của người dùng.
4. CDN giúp giảm độ trễ, tăng tốc độ truyền dữ liệu và giảm tải cho máy chủ gốc.
5. Nếu trình duyệt và CDN hỗ trợ HTTP/3, kết nối có thể nâng cấp lên QUIC/UDP 443 để giảm độ trễ và cải thiện trải nghiệm khi mạng không ổn định.

Kết luận: Client không kết nối trực tiếp tới server gốc của YouTube mà được DNS anycast + CDN định tuyến tới node gần nhất (thường nằm trong ISP của người dùng).

A.3. Các giao thức mạng được sử dụng

A.3.1. Danh sách các giao thức

Giao thức	cổng	Tài liệu đặc tả	Vai trò
HTTPS	443	RFC 9110	Trao đổi dữ liệu web an toàn
HTTP/2	443/TCP	RFC 9113	Truyền nhiều request trên một kết nối TCP
HTTP/3	443/UDP	RFC 9114	HTTP chạy trên nền QUIC
DNS	53/UDP & 53/TCP	RFC 1035	Phân giải tên miền
QUIC	443/ UDP	RFC 9000	Giao thức truyền tải hiện đại, giảm độ trễ
TLS 1.3	-	RFC 8446	Mã hóa dữ liệu đầu cuối

A.3.2. Giao thúc tầng giao vận: TCP và UDP

YouTube sử dụng cả TCP và UDP tùy theo giao thức được thương lượng giữa client và server.

Chức năng	Giao vận	Lý do
Tải trang HTML, JSON, thumbnail	TCP (hoặc QUIC/UDP)	Cần tin cậy, không mất mát
Tải segment video (DASH)	TCP hoặc QUIC/UDP 443	Ưu tiên tin cậy nhưng giảm độ trễ khi mất gói
Telemetry, log ẩn danh	UDP / QUIC	Không cần đảm bảo, giảm overhead
DNS	UDP 53 (fallback TCP 53)	Nhanh, gói nhỏ

Giải thích lựa chọn:

- TCP được sử dụng trong các kết nối HTTPS dựa trên HTTP/1.1 hoặc HTTP/2. TCP cung cấp cơ chế truyền dữ liệu đáng tin cậy, đảm bảo dữ liệu được nhận đầy đủ và đúng thứ tự. Điều này phù hợp với việc tải trang web, dữ liệu API và các đoạn video.
- Video on-demand không yêu cầu thời gian thực tuyệt đối như VoIP → TCP đảm bảo đủ byte để decode frame đúng thứ tự. Nếu dùng UDP thuần sẽ mất frame, hình bị vỡ.
- QUIC/UDP 443 (HTTP/3) cung cấp cơ chế truyền tin đáng tin cậy, hỗ trợ multiplexing và giảm thời gian thiết lập kết nối (0-RTT handshake). Nhờ đó QUIC giảm độ trễ và cải thiện trải nghiệm xem video trong điều kiện mạng không ổn định.
- TLS 1.3 bắt buộc để bảo vệ nội dung bản quyền, tránh ISP can thiệp và chèn quảng cáo.
- Vì vậy, không thể kết luận YouTube chỉ dùng TCP hoặc chỉ dùng UDP. Giao thức thực tế phụ thuộc vào trình duyệt, máy chủ và điều kiện kết nối mạng

A.4. Kiểu tương tác của ứng dụng

Hoạt động	Kiểu tương tác	Mô tả
Mở trang chủ	Request–Response	Client gửi yêu cầu và nhận HTML/JSON
Tìm kiếm video	Request–Response	Gửi yêu cầu tìm kiếm đến API
Tải metadata	Request–Response	Nhận tiêu đề, thumbnail, thông tin video
Phát video	Streaming	Video được chia thành nhiều đoạn nhỏ
Điều chỉnh chất lượng	Adaptive Bitrate	Client tự chọn chất lượng theo băng thông
Cập nhật dữ liệu	Polling/Push	Nhận hoặc kiểm tra dữ liệu mới

A.4.1. Streaming và Adaptive Bitrate

YouTube sử dụng cơ chế streaming để truyền và phát video theo từng đoạn nhỏ thay vì phải tải toàn bộ video về trước. Điều này giúp người dùng có thể bắt đầu xem video ngay trong khi các phần tiếp theo vẫn đang được tải xuống. Khi người dùng chọn một video, client sẽ nhận được manifest chứa

thông tin về các segment video với nhiều mức chất lượng khác nhau. Sau đó, client sẽ gửi yêu cầu để tải từng segment về và lưu vào buffer. Trong lúc một segment đang được phát, các segment tiếp theo cũng được tải xuống để đảm bảo video có thể phát liên tục.

Bên cạnh đó, YouTube sử dụng Adaptive Bitrate Streaming (ABR) để tự động điều chỉnh chất lượng video dựa trên tốc độ mạng và lượng dữ liệu hiện có trong buffer. Khi tốc độ mạng giảm hoặc buffer gần hết, hệ thống có thể chuyển sang chất lượng thấp hơn để hạn chế tình trạng video bị gián đoạn. Ngược lại, khi kết nối mạng ổn định, chất lượng video có thể được tăng lên. Nhờ cơ chế Streaming kết hợp với Adaptive Bitrate, YouTube có thể giảm tình trạng buffering và giúp người dùng có trải nghiệm xem video ổn định hơn.

A.4.2. Kiến trúc server và khả năng phục vụ đồng thời

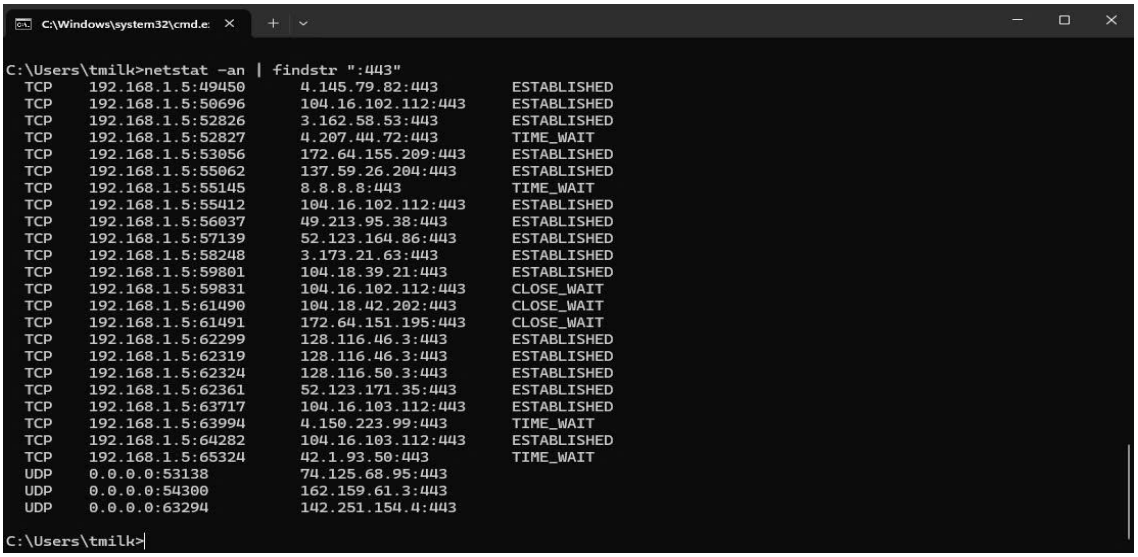
YouTube sử dụng kiến trúc Concurrent + Distributed:

- Load balancer (GSLB) phân phối yêu cầu theo vị trí địa lý
- CDN edge node phục vụ nội dung tĩnh và video
- Async I/O (epoll / io_uring) ở tầng backend Google
- Caching nội dung phổ biến để giảm tải máy chủ gốc

Ước lượng: YouTube phục vụ hàng tỷ giờ xem mỗi ngày → số client đồng thời lên đến hàng chục triệu phiên. Nếu server chỉ xử lý tuần tự, một yêu cầu tải video lớn có thể làm các client khác phải chờ → độ trễ cao và buffering. Vì vậy kiến trúc đồng thời là bắt buộc đối với nền tảng video quy mô lớn.

A.5. Quan sát thực tế

A.5.1. Quan sát bằng netstat



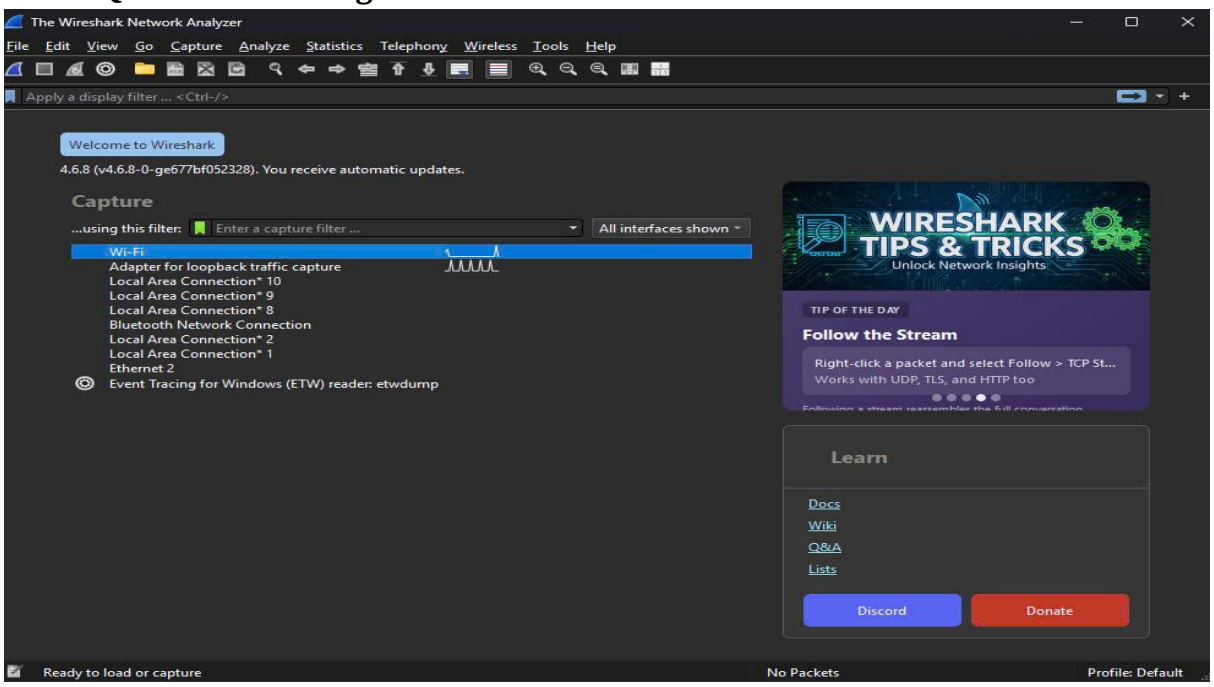
Hình A.5.1. Kết quả lệnh netstat khi truy cập YouTube.

Nhận xét: Qua kết quả kiểm tra, có thể thấy nhiều kết nối TCP sử dụng cổng 443 đang ở trạng thái ESTABLISHED hoặc TIME_WAIT. Điều này cho thấy máy tính

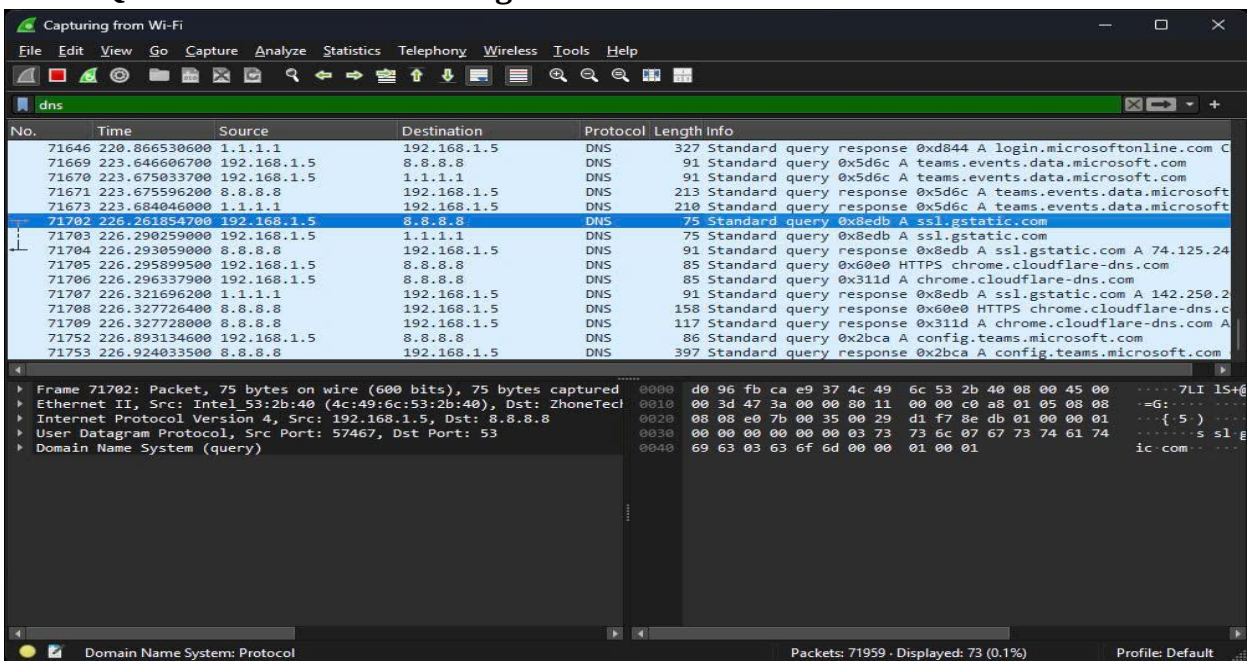
đang thực hiện các kết nối HTTPS để truyền và nhận dữ liệu trong quá trình xem video trên YouTube.

Ngoài ra, có một kết nối UDP sử dụng cổng 53 đến địa chỉ 8.8.8.8. Đây là kết nối được sử dụng cho quá trình DNS, giúp máy tính phân giải tên miền thành địa chỉ IP trước khi kết nối đến máy chủ.

A.5.2. Quan sát dns bằng wireshark



A.5.3. Quan sát HTTPS/TLS bằng wireshark



A.5.4. Nhận xét chung

- Địa chỉ IP server YouTube thay đổi theo vị trí địa lý → đặc trưng của CDN anycast

- Luôn thấy cổng 443 cho mọi lưu lượng chính (web, video, API).
- Cổng 53/UDP cho DNS.
- Không thấy nội dung video dạng plaintext vì đã được mã hóa bằng TLS 1.3 → chỉ phân tích được metadata.

Phần B - Thiết kế giao thức cho ứng dụng chat của nhóm

B.1. Mô tả ứng dụng

KetBan là một ứng dụng chat nhóm được xây dựng theo mô hình Client-Server. Người dùng kết nối đến Server để trao đổi tin nhắn với những người dùng khác trong hệ thống.

Các chức năng chính của ứng dụng:

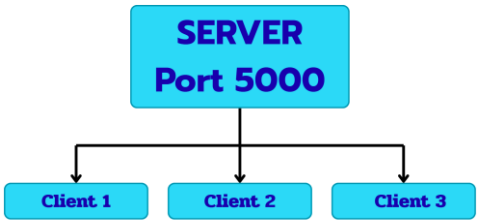
- Đăng nhập bằng tên: Người dùng nhập tên để tham gia hệ thống.
- Chat chung: Gửi tin nhắn cho tất cả người dùng đang online.
- Chat riêng: Gửi tin nhắn cho một người dùng cụ thể.
- Xem danh sách người dùng: Xem những người đang online.
- Thoát: Người dùng có thể thoát khỏi ứng dụng và đóng kết nối.

Ngoài các chức năng trên, nhóm có thể phát triển thêm một số chức năng như phòng chat, truyền tệp, lưu lịch sử trò chuyện và thông báo.

B.2. Kiến trúc

Ứng dụng sử dụng mô hình Client – Server. Server là thành phần trung tâm, chịu trách nhiệm tiếp nhận kết nối từ nhiều Client, xử lý yêu cầu và chuyển tiếp tin nhắn đến Client phù hợp.

- Giao thức giao vận:nhóm lựa chọn TCP (Transmission Control Protocol) vì ứng dụng chat cần truyền dữ liệu đầy đủ và đúng thứ tự giữa Client và Server.
- Cổng Server: 5000,Client sử dụng cổng này để kết nối đến Server.



Hình B.2 Sơ đồ kiến trúc hệ thống Client–Server

Server có thể nhận kết nối từ nhiều Client cùng lúc. Client gửi yêu cầu hoặc tin nhắn đến Server. Sau khi xử lý, Server trả kết quả hoặc chuyển tiếp tin nhắn đến Client tương ứng.

B.3. Đặc tả thông điệp

Các thông điệp trong ứng dụng được thiết kế dưới dạng văn bản và sử dụng UTF-8. Các trường dược phân cách bằng ký tự | và mỗi thông diệp kết thúc bằng \n.

Thông điệp	Hướng	Cú pháp	Ý nghĩa	Phản hồi
LOGIN	C → S	LOGIN|<tên>\n	Đăng nhập bằng tên	OK / ERR
MSG	C → S	MSG|<đích>|<nội dung>\n	Gửi tin nhắn chung hoặc riêng	ERR nếu lỗi
LIST	C → S	LIST\n	Yêu cầu danh sách người dùng	USERLIST
QUIT	C → S	QUIT\n	Thoát khỏi ứng dụng	BYE
OK	S → C	OK|<nội dung>\n	Thông báo yêu cầu thành công	-
ERR	S → C	ERR|<mã>|<nội dung>\n	Thông báo lỗi	-
USERLIST	S → C	USERLIST|<tên1>,<tên2>,...\n	MSG|<người gửi>|<nội dung>\n	-
MSG	S → C	MSG|<người gửi>|<nội dung>\n	Chuyển tin nhắn đến Client	-
BYE	S → C	BYE\n	Xác nhận Client thoát	Đóng kết nối

Ví dụ:
LOGIN|Nam\n
MSG|all|Xin chào mọi người\n

Cách viết LOGIN và MSG được xây dựng theo mẫu thông điệp trong đề bài.

B.4. Kịch bản trao đổi

Kịch bản 1: Đăng nhập thành công và gửi tin nhắn chung

Giả sử có hai Client là **Nam** và **Lan**

Thông điệp	Hướng	Cú pháp
LOGIN	C(a)→ S	LOGIN|Nam\n
OK	S → C(a)	OK|Chào Nam\n
LOGIN	C(b) → S	LOGIN|Lan\n
OK	S → C(b)	OK|Chào Lan\n
MSG	C(a) → S	MSG|all|Xin chào mọi người\n
MSG	S → C(a)	MSG|Nam|Xin chào mọi người\n
MSG	S → C(b)	MSG|Nam|Xin chào mọi người\n

Kết quả: Server nhận tin nhắn từ Nam và chuyển tin nhắn đến những Client đang online.

Kịch bản 2: Đăng nhập với tên đã tồn tại

1. Client A → Server:
   LOGIN|Nam\n
2. Server kiểm tra danh sách người dùng
3. Server phát hiện tên "Nam" đã tồn tại
4. Server → Client A:
   ERR|1|Tên đã tồn tại\n

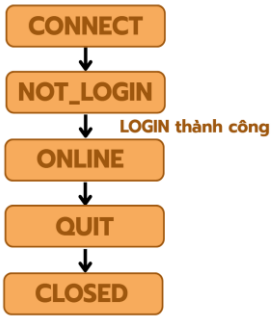
Client A không được chuyển sang trạng thái ONLINE và phải chọn một tên khác.

Ngoài ra, nếu người dùng gửi tin nhắn đến một người không tồn tại:

Client → Server:
MSG|An|Xin chào\n
Server → Client:
ERR|2|Người nhận không tồn tại\n

B.5. Quy tắc trạng thái và lỗi

Trạng thái của Client:



Hình B.5. Sơ đồ chuyển trạng thái của Client

Các quy tắc chính:

1. Khi Client kết nối đến Server, Client ở trạng thái NOT_LOGIN
2. Client phải đăng nhập thành công trước khi sử dụng chức năng chat
3. Khi ở trạng thái ONLINE, Client có thể gửi tin nhắn và yêu cầu danh sách người dùng
4. Khi muốn thoát, Client gửi thông điệp QUIT
5. Server gửi BYE và sau đó đóng kết nối
6. Nếu thông điệp sai cú pháp hoặc yêu cầu không hợp lệ, Server gửi ERR

Bảng mã lỗi

Mã lỗi	Ý nghĩa	Ví dụ
1	Tên người dùng đã tồn tại	ERR|1|Tên đã tồn tại\n
2	Người nhận không tồn tại	ERR|2|Người nhận không tồn tại\n
3	Thông điệp sai cú pháp	ERR|3|Sai cú pháp\n
4	Chưa đăng nhập	ERR|4|Chưa đăng nhập\n
5	Nội dung tin nhắn rỗng	ERR|5|Tin nhắn rỗng\n

Phần C - Thiết lập môi trường Python và chương trình đầu tiên

C.1. Môi trường thực hành

Nhóm thực hiện bài tập trên hệ điều hành Windows và sử dụng Visual Studio Code làm môi trường lập trình. Python được sử dụng với phiên bản từ 3.12 trở lên.

Thông tin môi trường của thành viên:

Thành viên 1

- Hệ điều hành: Windows 11
- Phiên bản Python: 3.14.7
- Công cụ lập trình: Visual Studio Code

Thành viên 2

- Hệ điều hành: Windows 11
- Phiên bản Python: 3.13.12
- Công cụ lập trình: Visual Studio Code

Thành viên 3

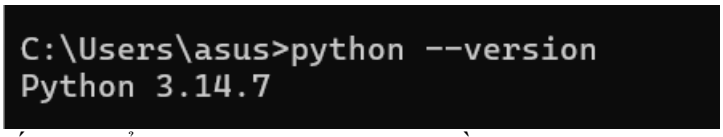
- Hệ điều hành: Windows 11
- Phiên bản Python: 3.14.7
- Công cụ lập trình: Visual Studio Code

Thành viên 4

- Hệ điều hành: Windows 11
- Phiên bản Python: 3.14.07
- Công cụ lập trình: Visual Studio Code

Thành viên 5

- Hệ điều hành: Windows 11
- Phiên bản Python: 3.13.2
- Công cụ lập trình: Visual Studio Code



Hình C.1. Kết quả kiểm tra phiên bản Python bằng lệnh **`python --version`**.

C.2. Chạy chương trình `hello_net.py`

Chương trình **`hello_net.py`** sử dụng mô-đun socket để lấy tên máy và các địa chỉ mạng được hệ điều hành cung cấp cho máy tính.

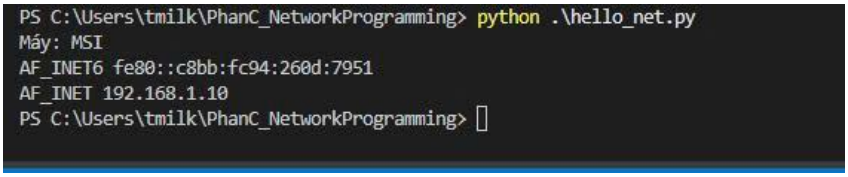
Mã nguồn chương trình:

`hello_net.py`

```
import socket

host = socket.gethostname()
print("Máy:", host)
for family, _, _, _, addr in socket.getaddrinfo(host, None):
    print(family.name, addr[0])
```

Sau khi chạy chương trình bằng lệnh: `python hello_net.py` chương trình hiển thị tên máy và các địa chỉ IP tương ứng.



Hình C.2. Kết quả chạy chương trình **`hello_net.py`**.

Nhận xét: Kết quả cho thấy chương trình có thể lấy được tên máy tính và các địa chỉ IP của từng máy. Địa chỉ IPv4 được sử dụng để tiếp tục kiểm tra và tính toán thông tin mạng.

C.3. So sánh kết quả với `ipconfig`

Nhóm sử dụng lệnh **`ipconfig`** để kiểm tra thông tin cấu hình mạng của máy tính. Kết quả IPv4 thu được được đối chiếu với địa chỉ hiển thị khi chạy chương trình **`hello_net.py`**.

Thông tin môi trường của thành viên:

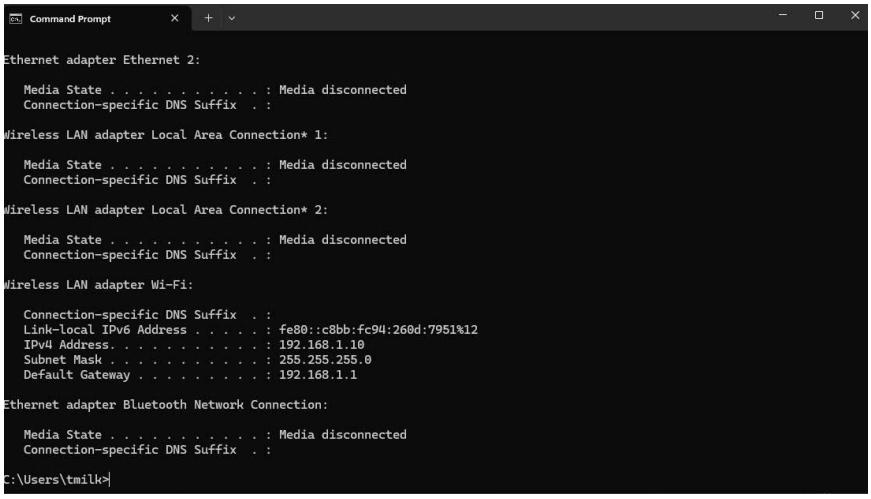
Thành viên 1
- IPv4 Address: 192.168.1.10
- Subnet Mask: 255.255.255.0

Thành viên 2
- IPv4 Address: 192.168.1.9
- Subnet Mask: 255.255.255.0

Thành viên 3
- IPv4 Address: 192.168.1.6
- Subnet Mask: 255.255.255.0

Thành viên 4
- IPv4 Address: 172.20.10.2
- Subnet Mask: 255.255.255.240

Thành viên 5
- IPv4 Address: 192.168.1.18
- Subnet Mask: 255.255.255.0



Hình C.3. Kết quả thực hiện lệnh **`ipconfig`**.

Nhận xét: Qua việc so sánh kết quả với lệnh "`ipconfig`", nhóm nhận thấy địa chỉ IPv4 và Subnet Mask lấy được từ máy tính là phù hợp với thông tin hiển thị trong hệ thống. Mỗi thành viên có địa chỉ IP khác nhau tùy theo mạng đang sử dụng. Kết quả này giúp nhóm hiểu rõ hơn về địa chỉ IP của máy và cách xác định mạng mà máy tính đang kết nối.

C.4. Chạy chương trình `dns_subnet.py`

Chương trình **`dns_subnet.py`** được sử dụng để phân giải tên miền thành địa chỉ IP và tính toán thông tin của mạng từ địa chỉ IP và subnet mask.

Mã nguồn chương trình:

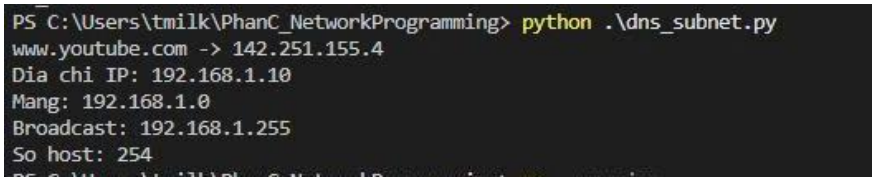
`dns_subnet.py`

```
import socket, ipaddress

domain = "www.youtube.com"
print(domain, "->", socket.gethostbyname(domain))

iface = ipaddress.ip_interface("192.168.1.9/24")
net = iface.network
print("Mạng:", net.network_address)
print("Broadcast:", net.broadcast_address)
print("Số host:", net.num_addresses - 2)
```

Trong đó IP_CUA_MAY/NETMASK được thay bằng địa chỉ IP và subnet mask thực tế của máy.
Sau khi chạy: **`python dns_subnet.py`**

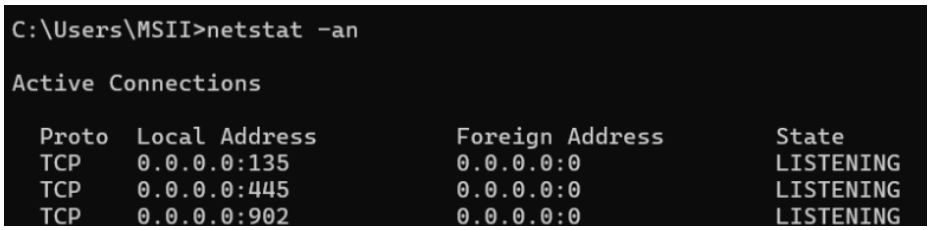


Hình C.4. Kết quả chạy chương trình **`dns_subnet.py`**.

Nhận xét: Chương trình đã phân giải tên miền thành địa chỉ IP và tính được địa chỉ mạng, địa chỉ broadcast và số lượng host của subnet.

C.5. Kiểm tra bằng netstat -an

Nhóm sử dụng lệnh **`netstat -an`** để liệt kê các kết nối mạng và xác định các cổng đang ở trạng thái LISTENING.
Lệnh thực hiện: **`netstat -an`**



Hình C.5. Kết quả kiểm tra các cổng đang LISTENING.

Ba cổng LISTENING được lựa chọn để tra cứu dịch vụ:

STT	Cổng	Giao thức	Trạng thái
1	135	TCP	LISTENING
2	445	TCP	LISTENING
3	902	TCP	LISTENING

Phần D – Câu hỏi thảo luận

Câu 1. Vì sao ứng dụng chat văn bản thường dùng TCP, còn gọi thoại / video thường dùng UDP? Nếu dùng ngược lại thì gặp vấn đề gì?

Giải: Ứng dụng chat văn bản thường sử dụng TCP vì tin nhắn cần được truyền đầy đủ, đúng thứ tự và không bị mất dữ liệu. TCP có cơ chế kiểm soát lỗi, truyền lại các gói tin bị mất và đảm bảo dữ liệu đến đúng thứ tự.

Trong khi đó, gọi thoại và video thường sử dụng UDP vì ưu tiên tốc độ và độ trễ thấp hơn việc đảm bảo mọi gói tin đều đến nơi. Nếu một vài gói âm thanh hoặc hình ảnh bị mất, cuộc gọi vẫn có thể tiếp tục và người dùng thường ít nhận thấy hơn.Nếu dùng UDP cho chat văn bản, tin nhắn có thể bị mất hoặc đến sai thứ tự. Ngược lại, nếu dùng TCP cho thoại/video, dữ liệu bị mất phải truyền lại có thể gây trễ, giật hoặc đứng hình, làm giảm chất lượng cuộc gọi.

Câu 2. Ứng dụng chat của nhóm có cần server đồng thời không? Mô tả điều gì xảy ra với người dùng thứ hai nếu server là server tuần tự.

Giải: Ứng dụng chat có nhiều client kết nối cùng lúc nên nên sử dụng server đồng thời (concurrent server). Server đồng thời có thể xử lý nhiều kết nối bằng cách tạo thread/process hoặc cơ chế tương tự cho từng client.

Nếu server là server tuần tự, server thường chỉ xử lý một client tại một thời điểm. Khi client thứ nhất đang kết nối và chờ nhận/gửi dữ liệu, server có thể bị giữ tại thao tác xử lý client này.

Khi client thứ hai kết nối, kết nối có thể phải chờ đến khi server xử lý xong client thứ nhất mới được phục vụ. Điều này khiến ứng dụng chat bị chậm hoặc người dùng thứ hai tưởng rằng server không hoạt động. Vì vậy, server chat nên được thiết kế theo mô hình đồng thời.

Câu 3. Khi hai client gửi thông điệp gần như cùng lúc, server phân biệt hai kết nối bằng thông tin nào? Vì sao hai client trên cùng một máy vẫn không bị lẫn?

Giải: Server phân biệt các kết nối dựa trên địa chỉ IP và số cổng (port) của socket. Với kết nối TCP, mỗi kết nối được xác định bởi bộ thông tin gồm:IP nguồn, port nguồn, IP đích và port đích.

Vì vậy, dù hai client chạy trên cùng một máy và có cùng địa chỉ IP, mỗi client vẫn sử dụng một port nguồn khác nhau. Server dựa vào thông tin này để xác định đúng client và dữ liệu tương ứng, nên các kết nối không bị nhầm lẫn.

Ví dụ**:** Hai client cùng kết nối đến server 127.0.0.1:5000, nhưng client 1 dùng port 50100, client 2 dùng port 50101. Server có thể phân biệt chính xác hai client.

Câu 4. Tin nhắn tiếng Việt đi qua socket cần được xử lý thế nào ở bên gửi và bên nhận? Điều gì xảy ra nếu hai bên dùng bảng mã khác nhau, hoặc nếu một thông điệp dài bị TCP chia thành hai lần recv()?

Giải: Tin nhắn tiếng Việt cần được mã hóa thành byte trước khi gửi qua socket và giải mã thành chuỗi ở phía nhận. Hai bên nên sử dụng cùng một bảng mã, phổ biến là UTF-8.

Ở bên gửi, chuỗi tiếng Việt được encode sang UTF-8 rồi gửi qua socket. Ở bên nhận, dữ liệu nhận được sẽ được decode bằng UTF-8 để khôi phục lại nội dung tiếng Việt.

Nếu hai bên sử dụng bảng mã khác nhau, nội dung có thể bị lỗi ký tự hoặc không thể giải mã chính xác.

Ngoài ra, TCP là giao thức hướng dòng (stream) nên một thông điệp dài có thể bị chia thành nhiều lần recv(). Vì vậy, chương trình cần có cơ chế xác định ranh giới của thông điệp, chẳng hạn dùng độ dài dữ liệu, ký tự kết thúc hoặc bộ đệm để ghép dữ liệu trước khi decode và xử lý.

KẾT LUẬN

Qua bài tập, nhóm đã có cơ hội tìm hiểu rõ hơn về hoạt động của một ứng dụng mạng như YouTube. Nhóm đã phân tích mô hình Client-Server, các giao thức mạng và cách dữ liệu được truyền giữa Client và Server.

Ngoài ra, nhóm đã thiết kế giao thức cho ứng dụng chat theo mô hình Client-Server, trong đó có các chức năng đăng nhập, gửi tin nhắn, xem danh sách người dùng và xử lý lỗi.

Thông qua phần thực hành, nhóm đã làm quen với Python, thư viện socket và một số lệnh mạng như ipconfig và netstat -an. Qua đó, các thành viên hiểu rõ hơn về cách các ứng dụng sử dụng mạng để trao đổi dữ liệu, cũng như cách kiểm tra và phân tích thông tin mạng trên máy tính. Nhìn chung, bài tập giúp nhóm củng cố kiến thức về mạng máy tính, giao thức truyền thông và lập trình socket, đồng thời nâng cao khả năng thực hành và làm việc nhóm.

TÀI LIỆU THAM KHẢO

[1] Internet Engineering Task Force (IETF), RFC 9293: Transmission Control Protocol (TCP), 2022.

[2] Internet Engineering Task Force (IETF), RFC 3629: UTF-8, a transformation format of ISO 10646, 2003.

[3] Python Software Foundation – The Python Standard Library: socket, ipaddress – https://docs.python.org/3/library/socket.html

[4] J. F. Kurose and K. W. Ross, Computer Networking: A Top-Down Approach, 8th ed., Pearson, 2021.

[5] B. Rhodes, J. Goerzen – Foundations of Python Network Programming, 3rd ed., Apress, 2014.

[6] K. R. Fall and W. R. Stevens, TCP/IP Illustrated, Volume 1: The Protocols, 2nd ed., Addison-Wesley Professional, 2011.

[7] Wireshark Foundation, Wireshark User's Guide, Version 4.7.4, available at: https://www.wireshark.org/docs/wsug_html/

PHỤ LỤC

Bảng phân công công việc

TT	Họ và tên	Mã sinh viên	Công việc đảm nhận	Đóng góp (%)
1	Phimmasone Khamphouvanh	3120224192	Nhóm trưởng: Phụ trách tổng hợp báo cáo, phân công công việc, kiểm tra và chỉnh sửa nội dung. Thực hiện Phần A: Phân tích ứng dụng mạng Youtube	20
2	Bounsavath Ameena	3120224190	Thực hiện Phần C: Thực hành công cụ mạng và Python, viết và kiểm tra chương trình dns_subnet.py và hỗ trợ kiểm tra mã nguồn, chụp và tổng hợp ảnh kết quả thực hành của các thành viên.	20
3	Lattanaphimphone Chindaphet	3120224191	Thực hiện Phần B: Thiết kế giao thức ứng dụng chat, xây dựng mô hình Client-Server và bảng mô tả các loại thông điệp, và hoàn thiện hình ảnh và định dạng báo cáo	20
4	Silaphet Thit	3120224188	Thực hiện Phần D: Câu hỏi thảo luận, tìm hiểu TCP, UDP, máy chủ đồng thời và cách xử lý dữ liệu UTF-8.	20
5	Thepsoulivong Syvone	3120224193	Thực hiện Phần C: Thực hành công cụ mạng và Python, viết và kiểm tra chương trình hello_net.py.	20